

0.72.2. Plus vite sur la grille : `deplace_croix` à 100 — doublé, deux positions

Deux lettres, la même croix : deux appels, dans la même image.

1. Ce qu'il faut comprendre

C'est le 0.72 et le 0.72.1 réunis : deux appels, un par lettre.

Les deux bougent dans la même image : la console ne fait passer le temps d'attente qu'une fois par image ; chaque appel de cette image a le droit de faire son pas.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace_croix>` → [0.66.3. La fonction `deplace\_croix\(\)` — suivre la croix, case par case](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1214 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	81 o	← la console, à l'allumage – la mise en route de la console
<code>deplace_croix()</code>	217 o	← <code>main()</code> – une fonction fournie par le langage
<code>main()</code>	57 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← <code>image()</code> – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AttendreAcces	12 o	← <code>EffacerCases</code> – attend de pouvoir écrire une case
AdresseCase	10 o	← <code>effacer()</code> , <code>poser()</code> – trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← <code>effacer()</code> – efface des cases
CopierTuiles	18 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
LireManette	37 o	← <code>bouton()</code> – lit les huit boutons
Tuiles	704 o	← <code>CopierTuiles</code> – la police entière

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

<code>bouton()</code>	× 4	20 o sur place, dans <code>deplace_croix()</code> + <code>LireManette</code> (37 o)
<code>effacer()</code>	× 1	15 o sur place, dans <code>deplace_croix()</code> + <code>AdresseCase</code> (10 o), <code>EffacerCases</code> (12 o)
<code>poser()</code>	× 1	13 o sur place, dans <code>deplace_croix()</code> + <code>AdresseCase</code> (10 o)
<code>image()</code>	× 1	3 o sur place, dans <code>main()</code> + <code>AttendreImage</code> (28 o)
<code>images()</code>	× 1	2 o sur place, dans <code>deplace_croix()</code>

Tes fonctions appelées :

<code>deplace_croix()</code>	× 2	38 o sur place, dans <code>main()</code>
------------------------------	-----	--

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <deplace_croix>    // une tuile qui suit la croix, case par case
#include <ALPHABET>         // les lettres de la police : ALPHABET[0] est le A
```

2. UNE VARIABLE — X

```
uint8_t x = 9;           // le A
```

3. UNE VARIABLE — Y

```
uint8_t y = 8;
```

4. UNE VARIABLE — XB

```
uint8_t xb = 4;          // le B
```

5. UNE VARIABLE — YB

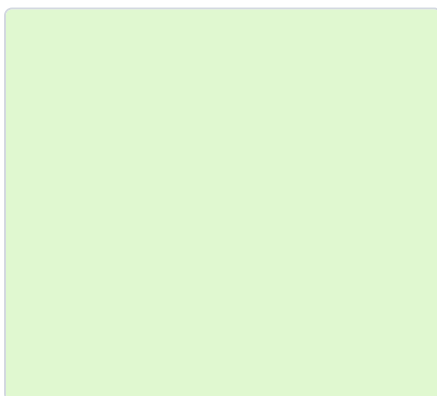
```
uint8_t yb = 4;
```

6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

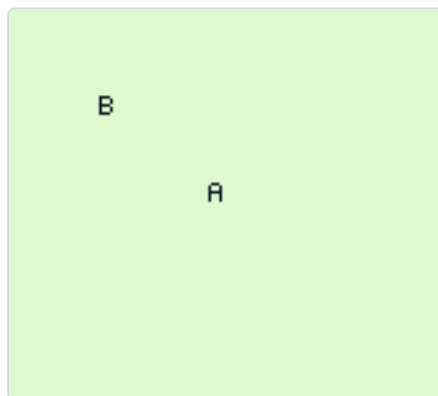
```
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 100);
        deplace_croix(xb, yb, ALPHABET[1], 100);
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Le A et le B suivent la croix ensemble.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A et le B ont fait 5 pas

Ensuite : **0.73. Glisser en une ligne : glisse_croix** — chapitre 0 : Avant tout. · `gameboy3` — écrire une cartouche Game Boy en C++